

Role gestion

II.1.1 Required Team Roles
Your team must assign the following roles (one person can have multiple roles if the team has 4 members):

- Product Owner (PO): Defines the product vision, prioritizes features, and ensures the project meets user needs.
- Maintains the product backlog.
- Makes decisions on features and priorities.
- Validates completed work.
- Communicates with stakeholders (evaluators, peers).
- Project Manager (PM) / Scrum Master: Facilitates team coordination and removes obstacles.
- Organizes team meetings and planning sessions.
- Tracks progress and deadlines.
- Ensures team communication.
- Manages risks and blockers.
- Technical Lead / Architect: Oversees technical decisions and architecture.
- Defines technical architecture.
- Makes technology stack decisions.
- Ensures code quality and best practices.
- Reviews critical code changes.
- Developers (all team members): Implement features and modules.
- Write code for assigned features.
- Participate in code reviews.
- Test their implementations.
- Document their work.

Team Size:

- 4 people: Some members will have multiple roles (e.g., PM + Developer, PO + Developer).
- 5 people: Roles can be more specialized, with dedicated PO, PM, Tech Lead, and 2 Developers.

All roles must be clearly documented in your README.md.

****A little look of what we would like:****

Nicolsan -> Project manager (likes visual things such as react for front)
Malapoug -> dev (do not want frontend) likes cpp
Ylabussi -> dev (do not want frontend)
Mm-furi. (would be ok for front/ts if i remember well?)
Yaoberso -> idk, to discuss what he'd like

Project Content

Project obligations

- Must be a web application
- it has a frontend, a backend and a database
- use clear and descriptive commits on github! yes, if thi isn't the case, **we can have ZERO for this!!!**
- We have to use a conteenerisation (without a doubt docker)
- We have to be able to start and run the entire project with a single command!!!
- We have to use the **last stable version of chrome** !!
- Absolutely no waring/error in the browser console!!
- We need to include Privacy Policy and Termes of Services page with relevant content (mean no place holder but real texts we need to make) --> more info: see index 1*
- We need to handle and Support multiple users at the same time ---->more info: see index 2*

Technicals requirements

STILL, THESE ARE OBLIGATIONS !!!

- We need to use a css framework (see help part 1)
- We needddd to Store credentials (API keys, environment variables, etc.) in a local .env file that is ignored by Git, and provide an .env.example file.
- Data base must be clear
- Have a user management system (password, mails, OAuth2.) --> more info: see index 3
- All forms and user inputs must be properly validated in both the frontend and backend.
- We need to use HTTPS!!!

MODULES

Cpp is the best way to implement games I think, and we may all be according to make a game I think ?
That's why I think cpp is best for back.

WEB:

- 1 frontend framework(react) 1pts
- 2 real time feature using web sockets 2pts
- 3 users interactions (chat, friends, profile) 2pts
- ORM may be? 1pts
- public API maybe ? 2 pts
- 4 notifications system (it seems easy) 1pts
- custom made design system with reusable component (may be the sprites can count ?) 1 pts
- 5 search functionality des in game work?1 pts
- 6 game statistics, history, etc... 1pts
- OAuth2 2.0 ? (genre log via github/google/42 etc...) 1pts
- organisations (like guilds in clash of clan ?) 2pts
- 2FA? 1pts
- Cybersec (I may wanna do this major module) 2pts
- 7 implemet a game (we need t have win/loss condition, so we need to pay attention to mmorpgs) 2pts
- 8 remote player (different computers)2pts
- 9 multiplayer (more than 2 player, basically a mmorpg) 2pts
- 0 game customization 1pts
- 1 gamification system (attentions to the size of it(warned in the subject))1pts
- 2 spectator mode 1pts

TOTAL: 17

MINDUSTRY LIKE

STACK

To determine, maybe learn go/rust ??

I'd like to use cpp for the backend, as we're now allowed to use current c++ (the 11 and 17 are so usefull)
Js/ts + react(nicolsan looked like to appreciate it) for the frontend

So the team yould be organized like this:

front:

- nicolsan on react
- Mm-furi on ts/js (as you already use often)

backend:

- Malapoug (I wanna do back, don't really like front, plus if it's in cpp, I love cpp)
- Ylabussi, same as he said he didn't like front (prefer cpp or anything else?)
- yaoberso, (I think it'd be preferable to have more people on the back than the front but if you prefer front we can see how to split the project like this)

Place holder

README.md

I can do it, I like doing it x)
But please keep track of everthing you do and use so it's easier made at the end of the project

Place holder

HELP

What is a Framework? For this project, a **framework** is defined as a comprehensive tool that provides:

- A structured architecture and conventions for organizing code.
- Built-in features for common tasks (routing, state management, etc.).
- A complete ecosystem of tools and libraries.

Examples:

- Frontend frameworks: React, Vue, Angular, Svelte, Next.js (these are frameworks).
- Backend frameworks: Express, Fastify, NestJS, Django, Flask, Ruby on Rails.
- Not frameworks: jQuery (library), Lodash (utility library), Axios (HTTP client).

Note: React is considered a framework in this context due to its ecosystem and architectural patterns, even though it is technically a library.

Evaluation part

Place holder

During evaluation, the team will be asked to explain:

- How roles were distributed.
- How work was organized and divided.
- How you communicated and coordinated as a team.
- How each member contributed to the project.

All team members must be able to explain the project and their contributions.

Use github issues
Make absolutely clear commits
try to have at least one other person reviewing important code change/implementation (maybe during weekly meetings ?)
Make absolutely clear comment in your code (so the review by other and comprehension will be easier)
If you touch the code from any other teammate, ask first, and above all, try to copy his way to code so he's not lost when he get his code back x)
try to keep track of when you implemented some of your important features, like in a to do list (may be with commits of when it has been done), keep documentation too (easier for the readme)